ATNLP: Lecture 16
Mechanistic Interpretability

Shay Cohen

February 23, 2026

School of informatics

Interpretability

- Neural networks achieve state-of-the-art results on many prediction tasks
- They have become more sophisticated and general
- At the moment, they are largely black boxes
- To increase human trust in them and avoid uninformed or risky use, we need to better understand their mechanism

Hence, the field of **interpretability**

Approaches to interpretability

To understand better a neural network model or a large language model, we can:

- Test its inputs and outputs and learn about the model
- Inspects its innerworkings (for example, activations, weights, etc.)

Analogous to the difference between a behavioural approach and a neuroscience approach to psychology?

Approaches to interpretability

- Behavioural: model is black-box, analysing inputs and outputs
- Attributional: explain outputs by tracing predictions to individual input contributions using gradients
- Concept-based: probe learned representations for high-level concepts; also includes representation engineering
- Mechanistic: study the components of models through analysis of features, neurons, layers and connections

Mechanistic interpretability

- Aims to completely specify a neural network's computation (reverse engineering)
- Focus on AI safety as a motivation

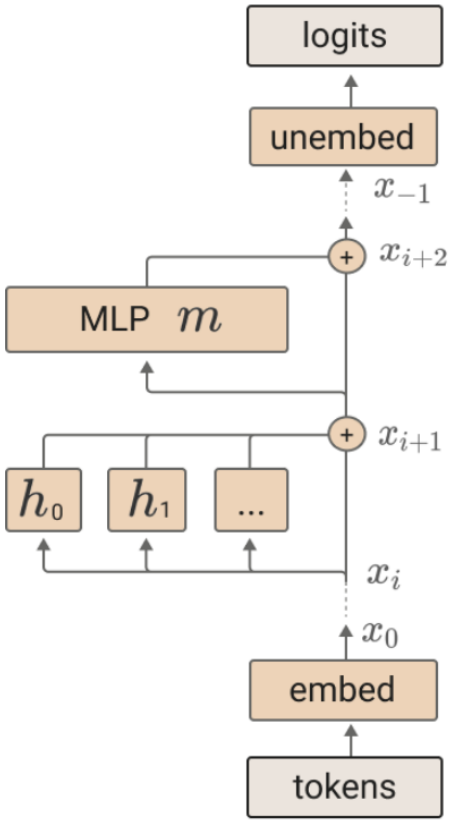
A super-mini crash course in linear algebra

We will use basic notions from linear algebra:

- **Vectors** - rather than thinking of them as a list of numbers, think of them as a direction with length
- **Dot product** - provides a similarity between two vectors; if one of the vectors are of length 1 (say v), the dot product $u \cdot v$ provides the (signed) length of projection of u onto v
- A set of vectors has a **span** - linear combination of all vectors (meaning, we sum scaled versions of the vectors in the set to get a new combined direction)
- A **basis** spans a subspace - all vectors in the basis are "linearly independent" and every vector in the subspace is a linear combination of the vectors in the basis

Some of this will become clearer later. There will be lots of new concepts potentially in this lecture, so it is fine if you miss some. See References (mostly the survey)

Reminder about neural networks and transformers



Reminder about transformers: layers, residual stream, activations...

The basic notion of features

- Features are the basic "constituent" in analysing transformer models or neural networks
- It is difficult to always identify them, but you can think of them as a "direction" that is activated given a specific type of input
- When we have a feature, we can "measure" how much an activation is aligned with it

Features

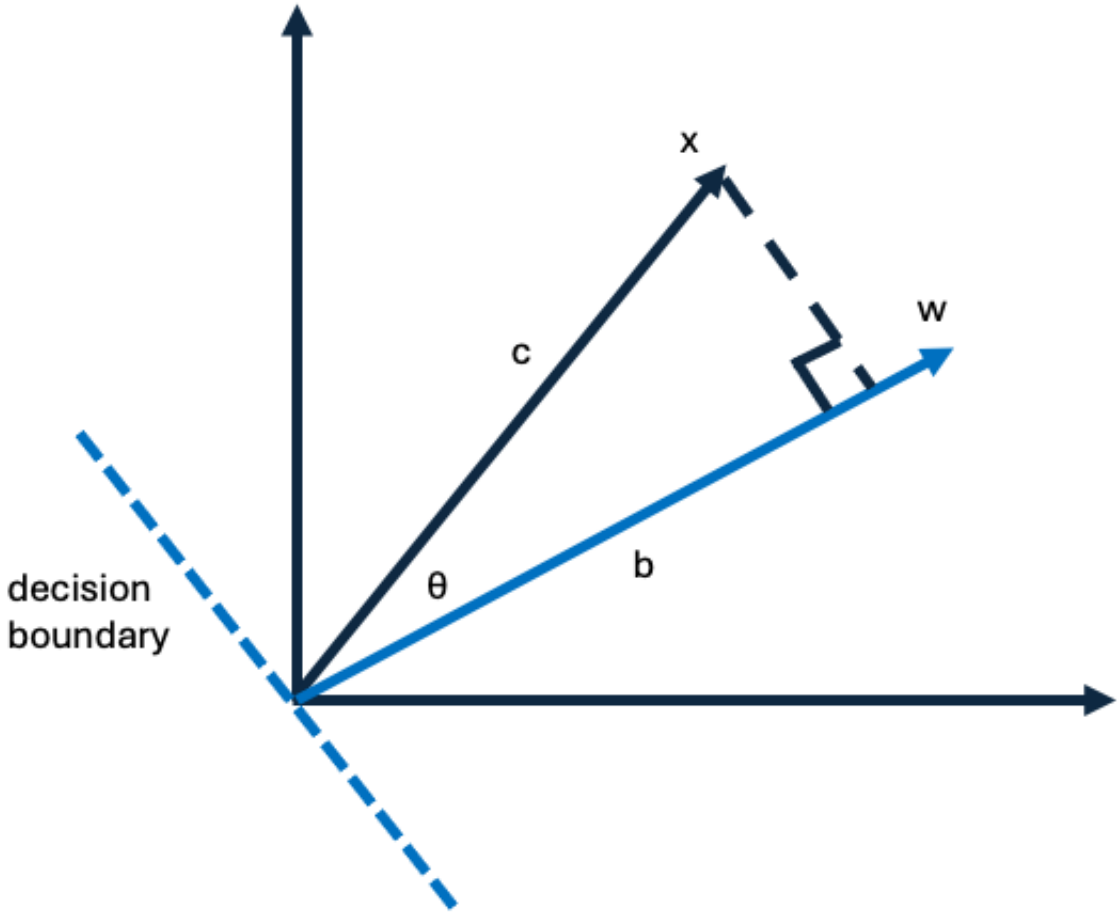
- Features are one of the fundamental and basic concepts in mechanistic interpretability
- When we identify a feature, we can see how similar activations are to it, and therefore how much they capture the feature
- The relationship between the neural network and the features can also be complex (polysemanticity and superposition)

Example process to calculate feature alignment

- Train a logistic regression model where the input is activation x and the target is whether a feature appears in the input or not (probe)
- Using the decision boundary vector for logistic regression w, calculate alignment with that feature
- A form related to "probing"

We will talk more about this later

Feature "alignment"



$\cos\theta = \frac{b}{c}, ||w|| = 1, \sum_i x_i w_i = c\cos\theta, c = ||x||$

Therefore, the dot product $\langle x, w \rangle$ provides (a) the distance from the decision boundary; (b) the length of the projection of x to w

More about features

- Features are assumed to be irreducible (cannot be decomposed further into features)
- This means it is not a linear or non-linear combination of more basic features - it is an "atom"
- Functionally, this means that if f is an irreducible feature, there is no "reasonable" function g such that $f = g(f_1, \ldots, f_n)$ for some other features f_i

Representation types

- Features - a "direction" in the activation space, for example, corresponding to "green pixels" or "upper case letter"
- Concepts - a grouping of features or some other pattern of features that corresponds more abstractly to a concept (leaf/tree, name of a city)

Monosemanticity and polysemanticity

- A neuron that fires for a single concept/feature is **monosemantic**
- It is **polysemantic** if it fires for multiple unrelated concepts/features (for example, images of cars and cats, or cities and words related to agriculture)

Food for Thought: What can we do with monosemanticity?

Monosemanticity and polysemanticity

- A neuron that fires for a single concept/feature is **monosemantic**
- It is **polysemantic** if it fires for multiple unrelated concepts/features (for example, images of cars and cats, or cities and words related to agriculture)

Food for Thought: What can we do with monosemanticity? This is useful, for example, if we want to neutralise "bad" behaviour

Privileged basis

An important notion with features is that of "privileged basis"

Features are correlated and intertwined, therefore we have lack of monosemanticity

A **privileged basis** is a choice of "coordinate system" that is more aligned with irreducible features (or aligns with other "semantics" the network computes), so that each axis corresponds to a meaningful feature

Neurons could become monosemantic in this case

Makes, for example, "intervention", as mentioned, easier

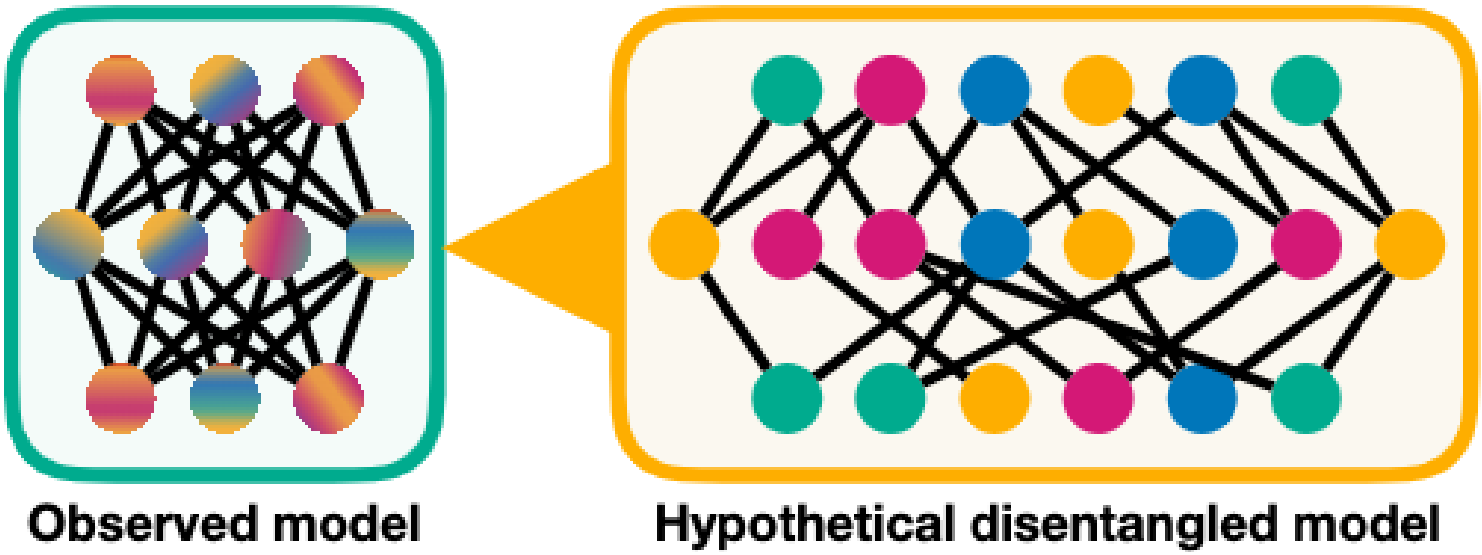
Polysemanticity

Polysemanticity is widespread. Why are some neurons polysemantic?

- Features do not "align" with neurons in a non-privileged basis
- Some believe that redundancy due to noise introduced during training encourages polysemanticity
- The idea of *superposition*: there is a limited number of neurons and many concepts

Superposition

The superposition hypothesis: Neural networks represent more features than they have neurons by encoding features in overlapping combinations of neurons



- Superposition: Multiple features are encoded in overlapping neuron directions, rather than each getting its own neuron (representation-level). $\Rightarrow$
- Polysemanticity: A neuron encodes multiple features / concepts at once (neuron-level).

A toy model (Elhage et al., 2022)

- Investigates the notion that a network can represent more features than neurons
- Let x be a high-dimensional vector, where x_i corresponds to a feature capturing real-world concept. Assumptions about features:
  - Concept sparsity (level of sparsity S)
  - More concepts than neurons
  - Some concepts are more important than others (denoted by a_i)
- Transform x into hidden representation h and reconstruct using x' as:

$$h = Wx;\ x' = \mathrm{ReLU}(W^\top h + b)$$

- Evaluate using a loss function

$$\mathcal{L} = \sum_x \sum_i a_i (x_i - x'_i)^2$$

A toy model

- Each x_i in the synthetic data is sampled based on S - the smaller the S, the more likely x_i (the feature) to be activated
- The matrix W transforms from the feature space to the "neuron" (representation) space ($rep \times feature$)
- Each row in W represents the "intensity" of the features for the corresponding row-indexed neuron. Each column provides a coverage of a feature in the neuron space.
- **Food for thought:** What does a dot-product between two rows of W indicate?

A toy model

- Each x_i in the synthetic data is sampled based on S - the smaller the S, the more likely x_i (the feature) to be activated
- The matrix W transforms from the feature space to the "neuron" (representation) space ($rep \times feature$)
- Each row in W represents the "intensity" of the features for the corresponding row-indexed neuron. Each column provides a coverage of a feature in the neuron space.
- **Food for thought:** What does a dot-product between two rows of W indicate? How similarly two neurons respond to features. If the row is sparse, the neuron is "more monosemantic".
- **Food for thought:** What does a dot-product between two columns of W indicate?

A toy model

- Each x_i in the synthetic data is sampled based on S - the smaller the S, the more likely x_i (the feature) to be activated
- The matrix W transforms from the feature space to the "neuron" (representation) space ($rep \times feature$)
- Each row in W represents the "intensity" of the features for the corresponding row-indexed neuron. Each column provides a coverage of a feature in the neuron space.
- **Food for thought:** What does a dot-product between two rows of W indicate? How similarly two neurons respond to features. If the row is sparse, the neuron is "more monosemantic".
- **Food for thought:** What does a dot-product between two columns of W indicate? How similarly two features are covered by neurons
- **Food for thought:** As we increase sparsity levels, will we have more superposition or less of it?

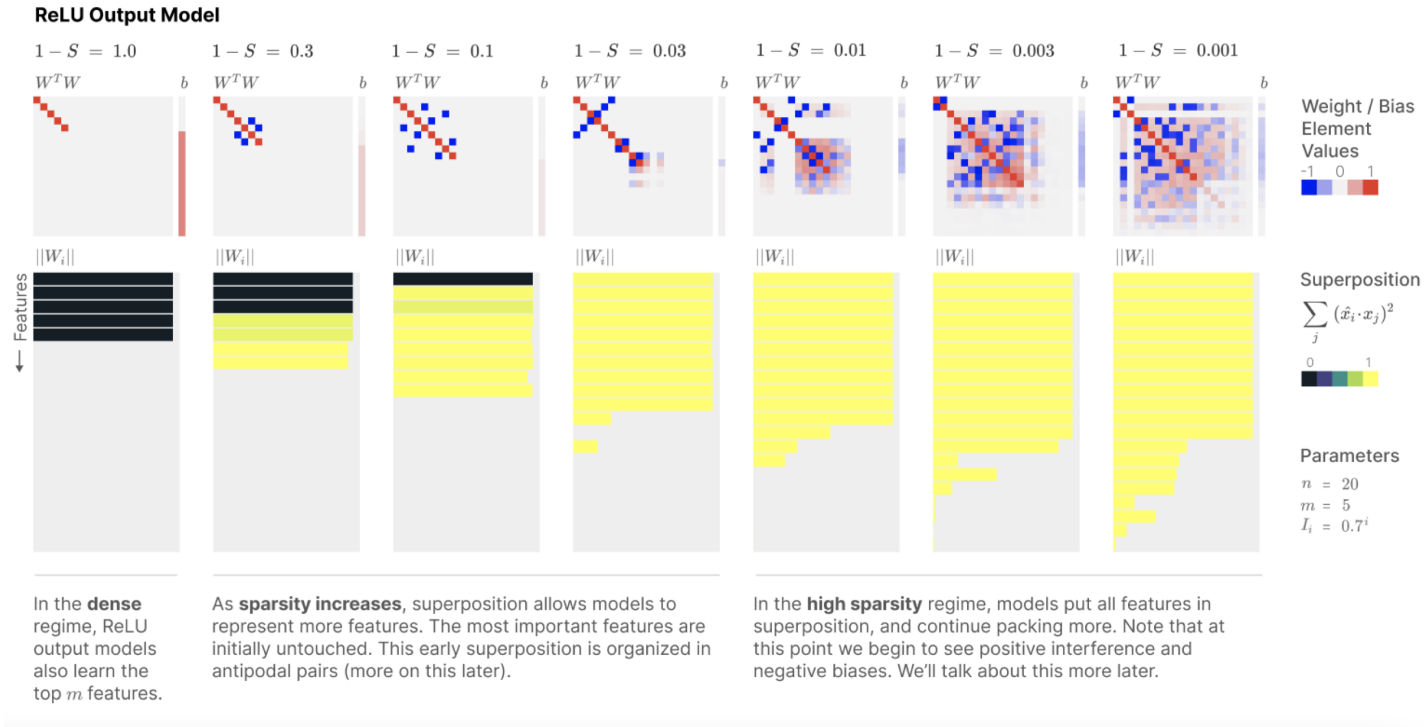
A toy model

- Each x_i in the synthetic data is sampled based on S - the smaller the S, the more likely x_i (the feature) to be activated
- The matrix W transforms from the feature space to the "neuron" (representation) space ($rep \times feature$)
- Each row in W represents the "intensity" of the features for the corresponding row-indexed neuron. Each column provides a coverage of a feature in the neuron space.
- **Food for thought:** What does a dot-product between two rows of W indicate? How similarly two neurons respond to features. If the row is sparse, the neuron is "more monosemantic".
- **Food for thought:** What does a dot-product between two columns of W indicate? How similarly two features are covered by neurons
- **Food for thought:** As we increase sparsity levels, will we have more superposition or less of it? Less interference among features, therefore more superposition (counter-intuitive?).

A toy model

- Each x_i in the synthetic data is sampled based on S - the smaller the S, the more likely x_i (the feature) to be activated
- The matrix W transforms from the feature space to the "neuron" (representation) space ($rep \times feature$)
- Each row in W represents the "intensity" of the features for the corresponding row-indexed neuron. Each column provides a coverage of a feature in the neuron space.
- **Food for thought:** What does a dot-product between two rows of W indicate? How similarly two neurons respond to features. If the row is sparse, the neuron is "more monosemantic".
- **Food for thought:** What does a dot-product between two columns of W indicate? How similarly two features are covered by neurons
- **Food for thought:** As we increase sparsity levels, will we have more superposition or less of it? Less interference among features, therefore more superposition (counter-intuitive?). If S is too high (extreme sparsity), features almost never fire, and less§ superposition - optimiser ignores dead features

A toy model



- $W^\top W$ - dot products between all feature weights (to neurons). The less diagonal a matrix is, the more similarly features are distributed across neurons. We see as features become more sparse ($1-S$ decreases), more superposition.
- $||W_i||$ corresponds to the norm of a column - how spread a given feature is across neurons. Yellow - very spread. The more yellow, the more features are spread (superposition)

Sparse autoencoders (Cunnigham et al., 2023)

Very similar approach to interpret different features:

- Sample the internal activations (residual stream / MLP sublayer / attention head sublayer)
- Train a sparse autoencoder with weights from a feature dictionary
- Use OpenAI's autointerpretability technique to name the features

Cunningham et al., 2023

- Transform x into x' using feature dictionary M:

$$c = \mathrm{ReLU}(Mx + b); \hat{x} = M^\top c = \sum_i c_i f_i$$

- f_i are the rows in the "feature dictionary" M
- c gives the coefficients for how much a given feature is used. Note that M has much more rows (the autoencoder features) than columns (the dimension of x)
- The objective to train the sparse autoencoder is $||x - \hat{x}||^2 + \alpha||c||_1$
- The use of ℓ_1 norm for c is what gives the sparsity. The first term is the "reconstruction loss"

Food for thought: What are we doing here, and how is that related to "privileged basis"?

Cunningham et al., 2023

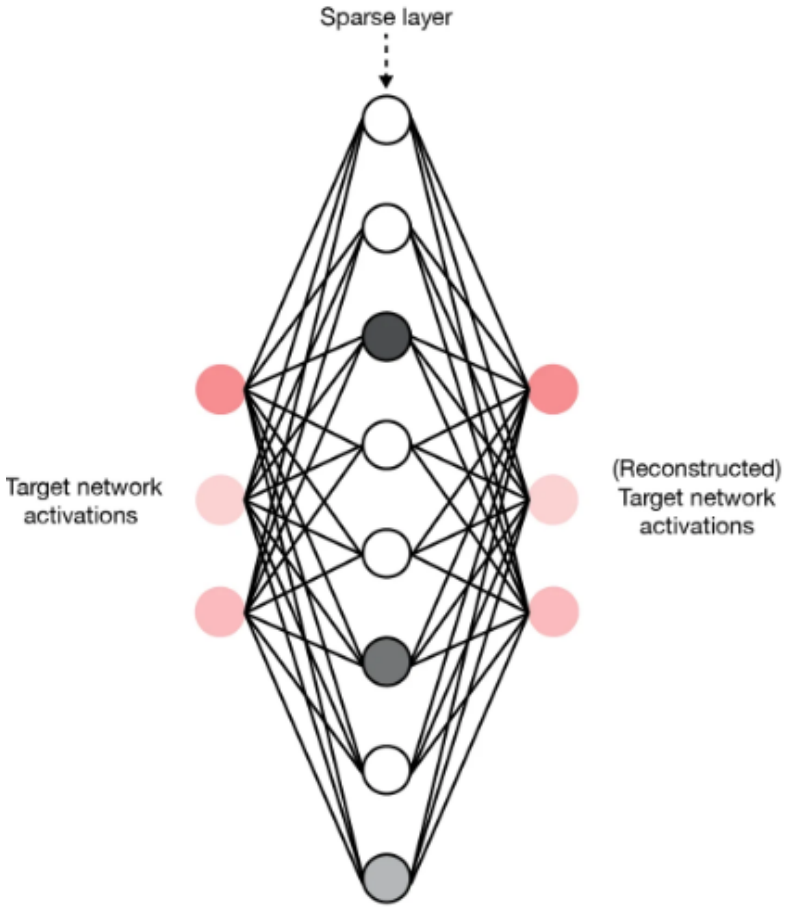
- Transform x into x' using feature dictionary M:

$$c = \mathrm{ReLU}(Mx + b); \hat{x} = M^\top c = \sum_i c_i f_i$$

- f_i are the rows in the "feature dictionary" M
- c gives the coefficients for how much a given feature is used. Note that M has much more rows (the autoencoder features) than columns (the dimension of x)
- The objective to train the sparse autoencoder is $||x - \hat{x}||^2 + \alpha||c||_1$
- The use of ℓ_1 norm for c is what gives the sparsity. The first term is the "reconstruction loss"

Food for thought: What are we doing here, and how is that related to "privileged basis"? We are trying to find a "privileged basis" with a larger dimension where each coordinate corresponds to some interesting coherent set of activations

SAEs



This is an example of a "shallow" SAEs. SAEs can be deeper.

We are trying to "invert" superposition

Some results

Feature	Description (Generated by GPT-4)	Interpretability Score
1-0000	parts of individual names, especially last names.	0.33
1-0001	actions performed by a subject or object.	-0.11
1-0002	instances of the letter 'W' and words beginning with 'w'.	0.55
1-0003	the number '5' and also records moderate to low activation for personal names and some nouns.	0.57
1-0004	legal terms and court case references.	0.19

Table 1: Results of autointerpretation on the first five features found in the layer 1 residual stream. Autointerpretation produces a description of what the feature means and a score for how well that description predicts other activations.

Circuits

Features are directions in the activation space.

Neural networks can be conceptualised as computational graphs, where **circuits** are sub-graphs consisting of linked features with weights

Circuits are the **computational primitive** and the networks' primary building block

Motifs

Patterns with circuits that repeat across tasks and models are called **motifs**

Examples:

- Curve detection in vision models (Cammarata et al., 2021; 2020)
- Circuits handling in-context learning (Olsson et al., 2022)

Motifs reveal the common mechanisms and strategies that emerge in neural architectures

Universality hypotheses

- **Weak:** While models converge to the same common underlying principles to solve given tasks, the specific features and circuits that implement these principles vary
- **Strong:** The same core features and circuits will universally arise in the form of a set of fundamental computational motifs

There is indeed evidence that ML models and neural networks tend to converge on similar features. Questions remain about the extent of this happening

Example Work on Universality

- "Universal dimensions of visual representation", Chen and Bonner (2024/2025), large empirical study showing a small set of shared latent dimensions across many vision models.
- "The Platonic Representation Hypothesis", Huh et al. (2024), a framing paper arguing representations are converging across models/modalities.
- "On Linear Identifiability of Learned Representations", Roeder et al. (2020/2021), theoretical results about identifiability up to linear transforms for discriminative models.
- "Similarity of Neural Network Representations Revisited", Kornblith et al. (2019), introduces/advocates CKA as a robust similarity measure and shows empirical layer correspondences.
- "Convergent Learning: Do different neural networks learn the same representations?", Yosinski et al. / Li et al. (2015–2018), early empirical investigations; shows both convergence and important differences.

Methods for mechanistic interpretability

There are two main classes:

- Methods that study feature representations and the model through **observation**

- Methods that do the same through **intervention**

This dichotomy is quite general, and applies to studying many other objects around us. However, there are various methods under these categories that are special to mechanistic interpretability

Observational methods

Several methods we will touch on (from the review paper):

- Structured probes
- Logit lens
- Sparse autoencoder (covered)

Structured probes

Food for thought: What's the issue with "regular" all-permissive probes?

Structured probes

Food for thought: What's the issue with "regular" all-permissive probes? When a probe is very flexible, it might indicate the "information is there", but it will not be really how the network uses it

Structured probes add constraints to probing so that the probe will work only if the information is encoded in a specific, interpretable form

For example, a sparsity constraint (related to SAEs); Or if you are interested in testing whether LLMs encode "syntax", enforce some parse tree constraints (close representations iff close trees)

Logit lens

Tries to answer: how would we know how different layers evolve in their prediction of tokens?

Take the last logit layer that predicts the tokens, and *apply it* on intermediate layers

You might detect, for example, where the network has developed a "response":

- Prompt: "Please complete, London is the capital of "
- Layer 3: high probability on country names, "France", "United Kingdom", "Canada", "Vietnam"
- Layer 5: perhaps more focus on European countries, probability of "United Kingdom" also increases
- Final layer: most of the probability mass on "United Kingdom"

Interventional methods

Several methods we will touch on (from the review paper):

- Activation patching
- Attribution patching
- Causal scrubbing

The main idea: you directly intervene in the process of inference to interpret some of its behaviour (rather than just observing and analysing activations post-hoc)

Activation patching

- Let M be a neural network and x and x' an input and a **counterfactual** input (to compare against)
- Let $r(x')$ be a latent representation from running M on x'
- Clamp part of the representation in M to part of $r(x')$ (in the corresponding place)
- Run again M on x

A "substitution" type of experiment

Food for thought: How do we use it?

Example

Let's say we suspect that a certain part of an LLM (in the representation of layer 3 somewhere) is in charge of subject-verb agreement (in number)

We take two inputs: *She is a singer* and *She are a singer*.

We run the first one, and then run the second one, clamping the part of the representation in layer 3 with the representation of the run on the correct sentence

If the prediction is now "is", this is strong evidence that this part of layer 3 is indeed dealing with subject-verb number agreement

Attribution patching

Very similar to activation patching, but also specifies a method to select which patch to replace

Example of attribution methods: integrated gradients, saliency maps, Smoothgrad, LIME/SHAP (we will not go into these methods)

Once we select where to patch, we repeat the same process as in activation patching

Causal scrubbing

- A way to test whether certain parts of a network are responsible for certain behaviours
- We start with an hypothesis about the network – which parts are responsible for a specific behaviour
- Main idea: divide the network into the parts that "matter" for the goal (according to an hypothesis) and those that should not
- Now follow "scrubbing": given an input, run it through, but for the parts that do not matter, randomise them all across and see if the output changes
- Do the randomisation many times
- If the output stays the same, this means that indeed the scrubbed-away information is irrelevant to the behaviour (hypothesis is correct)
- If the output changes, this means that these scrubbed parts do affect the behaviour (hypothesis was wrong)

Summary

- Mechanistic interpretability studies neural networks and provides an "interpretation" in terms of mechanisms in the network
- Most of the methods operate on the hidden activations in the network, whether observational or interventional
- "Features" are the basic units of discussion

References

Leonard Bereska Efstratios Gavves; Mechanistic Interpretability for AI Safety, A Review (2024)

Hoagy Cunningham et al.; Sparse Autoencoders Find Highly Interpretable Features in Language Models (2023)

We are currently working on a manuscript that is titled "the State of AI interpretability." If you are interested in a draft, send an email.